SYSTEM DESIGN

Database Sharding Visual Guide

Split one database into many:
scale writes past a single machine.

ANIMATED WALKTHROUGH
Step through pages to play the animation

STEP 1/6

Horizontal Partitioning

Rows are divided across shards; each shard is a full, independent DB.

App -> router -> shard(owner of key). Each shard holds a DISJOINT slice of rows.

- Vertical partitioning splits COLUMNS (rare). Sharding splits ROWS (the scaling move).
- Each shard has its own CPU, disk, and connections: write throughput multiplies by shard count.
- Trade-off: cross-shard joins and transactions get painful -- design the shard key first.

STEP 2/6

The Shard Key Decides Everything

Pick the column every query path can route on.

Hash key

shard = hash(user_id) mod S -- even spread, no range scans

Range key

shard by created_at or last name -- range queries stay local

Geo key

shard by region -- data lives near its users

Directory

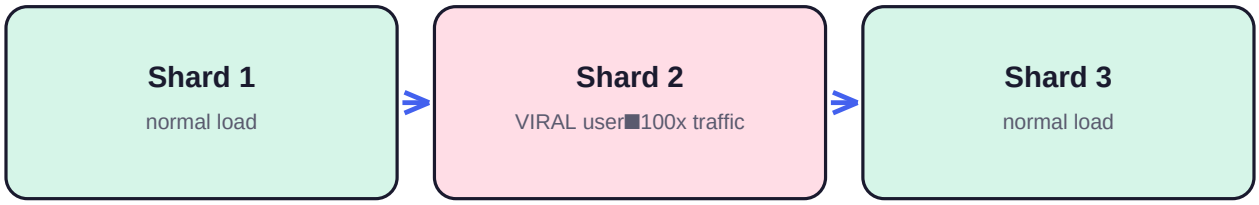
lookup service maps key -> shard -- flexible, extra hop

Good key: high cardinality, evenly distributed, present in most queries. user_id beats country_code.

STEP 3/6

Hotspots: The Celebrity Problem

Even hashing cannot save you when one KEY gets all the traffic.



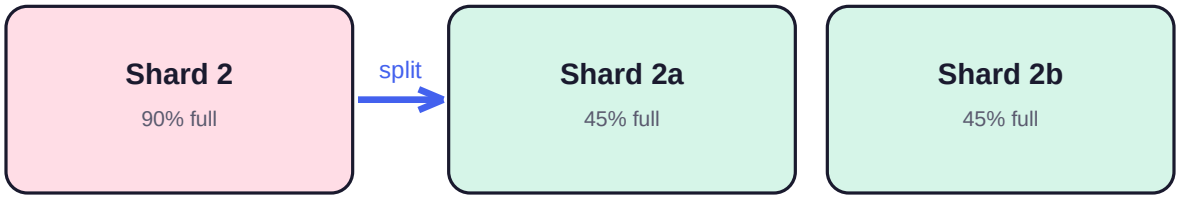
- A hot KEY overloads its shard while siblings idle -- adding shards does not help.
- Fixes: split the hot key (suffix buckets: user_123#1..#N), cache the hot read path heavily,
- or isolate hot tenants onto dedicated shards (noisy-neighbor quarantine).

Monitor per-shard QPS, not just the cluster average. Averages hide hotspots.

STEP 4/6

Rebalancing: Moving the Data

Shards fill unevenly over time; you must split or migrate ranges.



- Range shards: split the range in two, update the routing table. Hash shards: add a shard, remap (consistent hashing).
- Online migration: dual-write to old+new, backfill, then flip reads -- never stop the world.
- Vitess, Citus, and MongoDB's balancer automate this; DIY sharding means YOU build the mover.

STEP 5/6

Sharding vs Replication

They scale different things. Most systems need both.

Replication

copies of the SAME data■scales READS + availability

Sharding

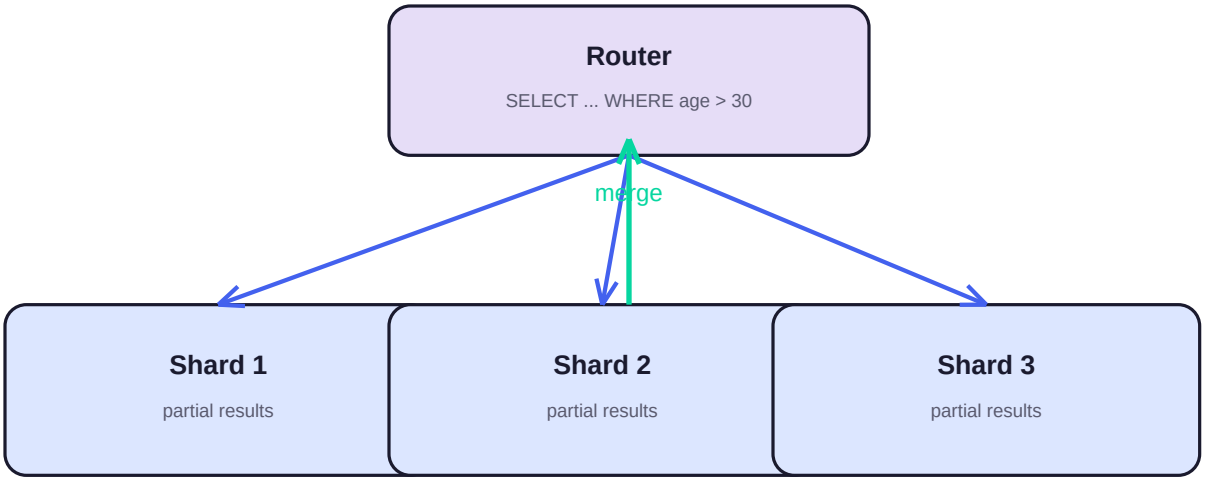
PARTS of the data per node■scales WRITES + capacity

- Read-heavy? Replicas (leader + followers) are simpler and keep joins intact.
- Write-heavy or data too big for one disk? Shard. Then replicate EACH shard for durability.
- Rule of thumb: replicate first (cheap), shard when one node cannot hold the write load.

STEP 6/6

Query Fan-Out (Scatter-Gather)

No shard key in the query? Ask every shard and merge.



Fan-out latency = SLOWEST shard. One straggler punishes every cross-shard query -- keep them rare.

STEP ✓I✓

Key Takeaways

Database Sharding — Remember

- Sharding splits ROWS across independent databases to scale writes and capacity.
- The shard key is the whole design: high-cardinality, even, present in hot queries.
- Hotspots come from hot KEYS, not bad hashing -- split, cache, or quarantine them.
- Rebalance online (dual-write + backfill); replicate each shard for durability.
- Replication scales reads, sharding scales writes; fan-out queries pay the slowest-shard tax.

Keep exploring

Pair this guide with the matching video walkthroughs and hands-on labs at interstitiumlabs.dev